

Examen blanc n°2 — Architecture & Exploitation Cloud Azure

Format : devoir sur table, 2 h, sans document ni ordinateur. **Barème :** /40. Mise en situation + questions de choix d'architecture et techniques. **Corrigé indicatif** en seconde partie — réponds d'abord seul, puis compare.

Énoncé — Cas « EduStream »

Conseil méthode : justifie **chaque** choix (coût / sécurité / disponibilité / exploitation). Une réponse non justifiée ne rapporte pas. « Ça dépend » est valable si tu dis **de quoi**.

Contexte. *EduStream* est une start-up qui propose une **plateforme web de formation en ligne** : cours, **vidéos pédagogiques**, quiz, espace étudiant. Les apprenants sont répartis en France et à l'international. L'usage est **très irrégulier** : faible la nuit, et de **gros pics** pendant les campagnes d'inscription et les périodes d'examens.

Existant. L'application tourne aujourd'hui sur **deux VM Azure** créées **à la main dans le portail**, derrière un Load Balancer. Les **vidéos** (plusieurs To, dont beaucoup d'anciennes peu consultées) sont stockées sur un **disque attaché à une VM**. La base est une **Azure SQL Database**. Il n'y a **qu'un seul environnement** (la prod) ; les développeurs testent directement dessus. Plusieurs personnes ont le rôle **Owner** sur la souscription. Aucun budget ni alerte de coût n'est configuré, et la facture grimpe.

Demande de la direction. Rendre la plateforme **élastique** (absorber les pics sans surpayer le reste du temps), **fiabiliser** (les coupures pendant les examens sont inacceptables), **séparer dev/test/prod** de façon reproductible, **maîtriser les coûts** du stockage vidéo, et **resserrer la sécurité**.

Partie 1 — Architecture & disponibilité (10 pts)

Q1. (3 pts) L'usage est très irrégulier. Quelle **propriété du cloud** exploiter, et **deux mécanismes Azure concrets** pour absorber les pics sans payer la capacité maximale en permanence ?

Q2. (3 pts) Les coupures pendant les examens sont inacceptables. Expliquez la différence entre **région** et **zone de disponibilité (AZ)**, et comment utiliser les AZ pour fiabiliser EduStream. Un Load Balancer **suffit-il** à garantir la haute disponibilité ? Justifiez.

Q3. (2 pts) Les apprenants sont **internationaux**. Citez **deux critères** qui doivent guider le choix de la (ou des) **région(s)** Azure.

Q4. (2 pts) Pour héberger l'application web élastique, recommanderiez-vous de **garder des VM** ou de passer à **Azure App Service** ? Donnez deux arguments et tranchez.

Partie 2 — Stockage vidéo & FinOps (8 pts)

Q5. (2 pts) Les vidéos sont sur un **disque de VM**. Pourquoi est-ce un **mauvais choix** pour ce type de données ? Quel service Azure proposez-vous à la place et pourquoi ?

- Q6.** (3 pts) Beaucoup de vidéos sont **anciennes et peu consultées**. Proposez une **politique de cycle de vie** du stockage pour réduire les coûts (mentionnez la notion de niveaux/tiers et ce qu'on en fait).
- Q7.** (3 pts) La facture grimpe sans contrôle. Citez **trois leviers FinOps** adaptés à EduStream (dont au moins un lié au caractère **prévisible** d'une partie de la charge, et un lié aux **environnements hors-prod**).
-

Partie 3 — Reproductibilité avec Terraform (8 pts)

- Q8.** (2 pts) Aujourd'hui les VM ont été créées **à la main**. Citez **deux problèmes** que cela pose pour séparer dev/test/prod, et comment l'**laC** les résout.
- Q9.** (3 pts) On veut déployer **dev, test et prod** à partir du **même code**, sans le dupliquer. Quels mécanismes Terraform mobiliser (citez-en au moins **trois** parmi variables, tfvars, locals, modules, outputs) et expliquez le rôle de chacun.
- Q10.** (3 pts) Un développeur a modifié **manuellement** une règle réseau dans le portail sur la prod. Au prochain **terraform plan**, que va-t-il se passer ? Comment s'appelle ce phénomène, pourquoi est-il **dangereux**, et quelle **règle d'équipe** proposez-vous ?
-

Partie 4 — Exploitation, supervision & sécurité (10 pts)

- Q11.** (2 pts) EduStream veut s'engager sur **99,9 % de disponibilité** auprès de ses clients écoles. Reliez cela aux notions de **SLI / SLO / SLA** avec un exemple pour chacune.
- Q12.** (2 pts) Proposez **deux alertes** utiles pour la plateforme (condition + ce qu'on fait quand elle se déclenche). Qu'est-ce que la « **fatigue d'alerte** » et comment l'éviter ?
- Q13.** (2 pts) Vous devez produire un **inventaire** des VM avec leur **taille** et leur **état**, rejouable et exportable. Pourquoi privilégier **Azure CLI** au portail ? À quoi sert l'option **--query** ?
- Q14.** (2 pts) Plusieurs personnes ont le rôle **Owner** sur la souscription. Quel **principe** est violé ? Quels **risques** ? Que proposez-vous concrètement ?
- Q15.** (2 pts) Un audit interne demande « **qui a modifié quelles ressources et quand ?** ». Quelle **source de traces** Azure répond à cette question, et que faut-il configurer pour **conserver** ces traces dans le temps ?
-

Partie 5 — Synthèse / Note DSI (4 pts)

- Q16.** (4 pts) Rédigez la **trame** (rubriques) d'une note de recommandations pour la direction d'EduStream et donnez **trois recommandations priorisées et justifiées** couvrant **trois domaines différents** (ex. disponibilité, coût, sécurité).

Corrigé indicatif

Attendus ; d'autres formulations justifiées sont acceptées. Points clés en gras.

Partie 1

Q1 — Élasticité. La propriété est l'**élasticité** : ajuster les ressources à la demande. Mécanismes : **autoscaling** (App Service Plan ou **Virtual Machine Scale Set** qui ajoute/retire des instances selon une métrique, ex. CPU ou nombre de requêtes) ; et/ou **dimensionnement dynamique** + arrêt des ressources hors-pic. → on paie la grosse capacité **uniquement** pendant les pics. (Bonus : mise en cache / CDN pour les vidéos.)

Q2 — Région vs AZ + HA. **Région** = zone géographique (latence, conformité, résidence des données). **AZ** = datacenters **physiquement séparés** dans une région (alimentation/réseau indépendants). Pour EduStream : déployer les instances web **sur plusieurs AZ** (+ base et LB zone-redundant) → résiste à la panne d'un datacenter. **Un LB seul ne suffit pas** : la HA = **plusieurs instances + plusieurs zones + sondes de santé + sauvegardes + supervision** ; un LB qui ne répartit que sur des VM d'une même zone reste vulnérable à la panne de cette zone.

Q3 — Choix de région. Critères : **latence** vers les utilisateurs (proximité géographique), **résidence/conformité des données** (RGPD pour l'UE), disponibilité des services voulus, et coût. (Pour de l'international : éventuellement plusieurs régions + un routage global type Front Door — bonus.)

Q4 — VM vs App Service. App Service : **autoscaling managé** + moins d'admin OS/patches → très adapté à une charge élastique et à une petite équipe. VM : **contrôle total** (utile si dépendances système spécifiques). Pour une plateforme web élastique → **App Service** (ou VMSS si on tient aux VM) est le meilleur choix.

Partie 2

Q5 — Disque VM → mauvais choix. Un disque attaché à une VM est **lié à cette VM**, peu scalable, peu durable pour des To de fichiers, non partagé et non adapté à la diffusion. → **Storage Account / Blob Storage** (objet) : durable, capacité quasi illimitée, accès indépendant des VM, intégrable à un CDN, et **niveaux de coût** différenciés.

Q6 — Cycle de vie. Blob propose des **niveaux (tiers)** : **Hot** (accès fréquent), **Cool** (peu fréquent), **Archive** (rare, très peu cher). Politique : déplacer automatiquement les vidéos vers **Cool après N jours** sans accès, puis **Archive** au-delà (ex. > 180 j), et éventuellement supprimer après la durée de rétention. → réduit fortement le coût du stockage des anciennes vidéos.

Q7 — FinOps (3 leviers). (1) **Reservations / Savings Plans** sur la **charge de base** stable (la plateforme tourne toujours un minimum) → réduction vs paiement à la demande. (2) **Éteindre / deallocate** les environnements **dev/test** hors usage (et l'autoscaling pour la prod). (3) **Tags obligatoires + Budgets + cost alerts** (et cycle de vie du stockage de la Q6) pour ventiler et détecter les dérives. (Advisor pour le rightsizing = bonus.)

Partie 3

Q8 — Création manuelle. Problèmes : (1) **non reproductible** → dev/test/prod ne seront jamais identiques (écarts, bugs « ça marche en test pas en prod ») ; (2) **non tracé / non versionné** → on ne sait pas qui a créé quoi. L'**laC (Terraform)** décrit l'état cible en code versionné → on recrée chaque environnement **à l'identique** et de façon **traçable**.

Q9 — Multi-env même code. Au moins trois : **variables** (paramètres d'entrée : région, taille de VM, nb d'instances) ; **terraform.tfvars** par environnement (valeurs concrètes : **environment=prod**, **vm_size=...**) ; **locals** (valeurs calculées : préfixe de nommage, tags communs) ; **modules** (factoriser réseau/compute réutilisés par les 3 envs) ; **outputs** (exposer IP/URL). → un seul code, trois jeux de valeurs = **zéro duplication, zéro divergence**.

Q10 — Drift. Terraform va **détecter l'écart** entre le code et l'état réel et proposer de **revenir à l'état déclaré** (ou il faut intégrer le changement au code). C'est le **drift**. Dangereux car l'infra devient **imprévisible**, la **sécurité s'affaiblit** et on perd la confiance dans le code comme source de vérité. Règle d'équipe : **tout changement passe par le code** (pull request + plan + apply), le **portail sert à observer/diagnostiquer**, pas à modifier. (Bonus : Azure Policy pour encadrer.)

Partie 4

Q11 — SLI/SLO/SLA. **SLA** = l'**engagement contractuel** de 99,9 % envers les écoles (avec pénalités). **SLO** = objectif **interne** plus strict (ex. 99,95 %) pour garder une marge. **SLI** = l'**indicateur mesuré** réel (ex. taux de disponibilité HTTP ou % de requêtes réussies). Relation : le SLI **mesure**, le SLO **vise**, le SLA **engage**.

Q12 — Alertes + fatigue. Ex. : (1) **disponibilité HTTP < 99,9 % sur 5 min** → escalade immédiate (incident) ; (2) **CPU/instances saturées pendant un pic** → vérifier l'autoscaling / scaler. La **fatigue d'alerte** = trop d'alertes non prioritaires → l'équipe les ignore et rate l'incident réel. On l'évite en ne gardant **en alerte que l'actionnable** (le reste en dashboard), en hiérarchisant (critique/avertissement/info) et en associant chaque alerte critique à un **runbook**.

Q13 — CLI + --query. CLI : commandes **rejouables, documentables, scriptables, exportables** (vs clic non reproductible du portail). **--query (JMESPath) filtre et met en forme** la sortie JSON (ex. ne garder que `name`, `vmSize`, `powerState`) → directement réutilisable dans un rapport ou un script.

Q14 — Owner multiple. Viole le **principe du moindre privilège**. Risques : **surface d'attaque** énorme (un compte compromis = contrôle total + facturation), **modifications/suppressions non maîtrisées**, perte de traçabilité. Proposition : **RBAC minimal** (Contributor sur le RG concerné, Reader pour l'observation, Owner réservé à 1-2 personnes), **revue d'accès** régulière, **MFA**.

Q15 — Audit. L'**Activity Log** répond à « qui a fait quoi et quand » (opérations de gestion). Par défaut sa rétention est limitée → configurer des **Diagnostic settings** pour **exporter** vers un **Log Analytics Workspace** (ou Storage) afin de **conserver et interroger** les traces dans le temps. (Entra ID logs pour les connexions = complément.)

Partie 5

Q16 — Note DSI. Rubriques : **Contexte · Constats · Risques · Recommandations priorisées · Plan d'action (court/moyen/long) · Indicateurs de suivi**. Trois recos, 3 domaines, ex. : - **Disponibilité (Haute)** : déploiement **multi-zone + autoscaling** → supprime le risque de coupure pendant les examens. - **Coût (Haute)** : **cycle de vie du stockage vidéo + réservations + budgets/alertes + extinction des env dev** → réduit la facture sans dégrader le service. - **Sécurité (Haute)** : **RBAC minimal** (retirer les Owner superflus) + **MFA** + export de l'**Activity Log** → réduit la surface d'attaque et assure la traçabilité.

Ce que ce sujet ajoute par rapport au n°1

Notion testée ici	Partie
Élasticité, autoscaling, VMSS	Q1, Q4
Région vs Availability Zone, HA multi-zone	Q2
Choix de région (latence, résidence des données)	Q3

Notion testée ici	Partie
Stockage objet + tiers Hot/Cool/Archive + cycle de vie	Q5, Q6
FinOps avancé : reservations , env hors-prod	Q7
Terraform multi-environnement (variables/tfvars/locals/modules)	Q9
Drift & règle d'équipe	Q10
Audit / Activity Log / rétention des logs	Q15
RBAC / moindre privilège (Owner multiple)	Q14

Réflexe d'examen (rappel) : structure chaque réponse en **constat** → **choix** → **justification (coût / sécurité / disponibilité / exploitation)** = la grille du Well-Architected Framework.